

BigQuery Avanzado & Analytics Engineering

En este sprint final, se rediseña la arquitectura de los datos para optimizar costes (Partitioning), se introduce lógica analítica avanzada (Window Functions) y se modela estructuras complejas (Arrays) para cerrar el ciclo con la automatización.

NIVEL 1	2
EJERCICIO 1	2
EJERCICIO 2	2
EJERCICIO 3	2
EJERCICIO 4	2
NIVEL 2	2
EJERCICIO 1	2
EJERCICIO 2	2
EJERCICIO 3	2
EJERCICIO 4	2
NIVEL 3	2
EJERCICIO 1	2
EJERCICIO 2	2
EJERCICIO 3	2

NIVEL 1

EJERCICIO 1

```
Sprint4_N1EJ1_Consulta_Tablas... Ejecutar Guardar
```

```
1
2 SELECT *
3 FROM `sprint3-analytics-irina.sprint3_silver.transactions_clean` AS T
4 JOIN `sprint3-analytics-irina.sprint3_silver.companies_clean` AS C
5 ON T.business_id = C.company_id
6 WHERE DATE(T.transaction_timestamp) = '2022-03-12'
7 AND C.country = 'Germany';
8
```

✓ Esta consulta procesará 10.87 MB cuando se ejecute.

Full Table Scan.

Etapas	Tiempos de trabajo				Filas
▶ S00: Input	Espera: 2 ms	Lectura: 32 ms	Procesamiento: 7 ms	Escritura: 4 ms	Registros leídos: 100 Registros escritos: 8
▶ S02: Coalesce	Espera: 2 ms	Lectura: 0 ms	Procesamiento: 8 ms	Escritura: 11 ms	Registros leídos: 8 Registros escritos: 8
▼ S03: Output	Espera: 5 ms	Lectura: 14 ms	Procesamiento: 20 ms	Escritura: 5 ms	Registros leídos: 100008 Registros escritos: 5

```
$1:business_id, $2:timestamp
FROM sprint3-analytics-irina.Sprint3_bronze.transactions_raw_native
WHERE equal(date($2), 19063)
$10, $11
FROM _stage02_output
```

Conclusión: Para ejecutar esta consulta, BigQuery realizó un Full Table Scan, leyendo más de 100,000 registros de la tabla transactions raw native sólo para devolver 5 filas de resultado. Esto es ineficiente porque se procesa 100% de los datos disponibles para obtener un resultado mínimo. Bigquery cobra por byte leído, como lee toda la tabla y no solo los filtros especificados, se gasta recursos innecesariamente.

EJERCICIO 2

```
Sprint4_N1EJ2_ReArquitectura  Ejecutar  Guardar consulta  Descargar

1
2
3 CREATE OR REPLACE TABLE `sprint3-analytics-irina.sprint3_silver.transactions_recent` AS
4 SELECT * EXCEPT(transaction_timestamp),
5     TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL CAST (FLOOR(RAND()*50) AS INT64)DAY
6 )AS Sales_Timestamp
7 FROM `sprint3-analytics-irina.sprint3_silver.transactions_clean`;
8
9
10 -- comprobación de la tabla
11 SELECT COUNT(*) AS fuera_rango
12 FROM sprint3_silver.transactions_recent
13 WHERE Sales_Timestamp <
14     TIMESTAMP_SUB(
15         CURRENT_TIMESTAMP(),
16         INTERVAL 50 DAY
17     )
18 OR Sales_Timestamp > CURRENT_TIMESTAMP();
19
20
21 -- código DDL
22 DROP TABLE IF EXISTS `sprint3-analytics-irina.sprint3_gold.fact_transactions_optimized`;
23
24 CREATE OR REPLACE TABLE `sprint3-analytics-irina.sprint3_gold.fact_transactions_optimized`
25 PARTITION BY DATE (Sales_Timestamp)
26 CLUSTER BY business_id
27 AS
28 SELECT *
29 FROM `sprint3-analytics-irina.sprint3_silver.transactions_recent`;
30
```

☆ fact_transactions_optimized 🔍 Consulta

ⓘ Esta es una tabla particionada. [Learn more](#)

Esquema	Detalles	Vista previa	Explorador
Descripción			
Etiquetas			
Claves primarias			
Etiquetas			
Tipo de tabla	Particionada		
Particionada por	DAY		
Particionada en el campo	Sales_Timestamp		
Vencimiento de la partición	Las particiones no vencen		
Filtro de partición	No es obligatorio		
Agrupada por	business_id		

En este ejercicio, el objetivo principal fue rediseñar la arquitectura de almacenamiento para evitar la caducidad de los datos y mantenerlos eficientes. Ya que debido a las limitaciones del BigQuery Sandbox, las tablas y particiones caducan automáticamente a los 60 días

Primero he creado una tabla intermedia con datos simulados (mocking data). Para ello, utilicé la función `TIMESTAMP_SUB` junto con `RAND()` para generar marcas de tiempo aleatorias dentro de los últimos 50 días. Apliqué la función `FLOOR` para redondear el resultado de la multiplicación y asegurar que el desplazamiento de días fuera un número entero válido antes de convertirlo con `CAST`. Finalmente, materialice la tabla en la capa Gold aplicando dos estrategias de optimización física:

Particionamiento por día sobre `Sales_Timestamp`: Para permitir que BigQuery ignore particiones irrelevantes y reduzca los bytes procesados en filtros temporales

Clustering por `business_id`: Para acelerar las consultas que filtran por empresa

EJERCICIO 3

Tabla original

```
-- Dry Run en la tabla original
SELECT *
FROM `sprint3-analytics-irina.sprint3_silver.transactions_recent`
WHERE DATE(timestamp) >= DATE_SUB(CURRENT_DATE(), INTERVAL 30 DAY);|
```

✓ Esta consulta procesará 11.85 MB cuando se ejecute.

Tabla optimizada

```
-- Dry Run en la tabla optimizada
SELECT *
FROM `sprint3-analytics-irina.sprint3_gold.fact_transactions_optimized`
WHERE DATE(Sales_Timestamp) >= DATE_SUB(CURRENT_DATE(), INTERVAL 30 DAY);
```

✓ Esta consulta procesará 7.34 MB cuando se ejecute.

He realizado una prueba de ejecución (Dry Run) comparando la eficiencia de ambas tablas mediante una consulta de selección total filtrada por los últimos 30 días.

Tabla No Optimizada: 11,85 MB procesados.

Tabla Optimizada: 7,34 MB procesados.

Cálculo del ahorro: $((11,85 - 7,34) / 11,85) * 100 = 38,06\%$

Con la tabla optimizada se ahorra un 38,06% en los bytes procesados.

EJERCICIO 4

Sprint4_N1EJ4_Vista Materializ...

Ejecutar

Guardar consulta

```
1
2 -- Crear la Vista Materializada con Particionamiento.
3
4 CREATE MATERIALIZED VIEW `sprint3-analytics-irina.sprint3_gold.mv_daily_sales`
5 AS
6 SELECT
7   DATE(Sales_Timestamp) AS sales_date,
8   COUNT(*) AS total_transactions,
9   SUM(amount) AS total_sales,
10  FROM `sprint3-analytics-irina.sprint3_gold.fact_transactions_optimized`
11  WHERE declined = 0
12  GROUP BY 1;
13
14 -- Consulta vista materializada
15 SELECT * FROM `sprint3-analytics-irina.sprint3_gold.mv_daily_sales`;
```

✓ Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Información del trabajo	Resultados	Visualización	JSON	Detalles de la ejecución
Fila	sales_date	total_transactions	total_sales	
1	2026-05-27	1924	497979.6200000...	
2	2026-04-25	1928	498976.5500000...	
3	2026-04-26	1937	493288.8699999...	
4	2026-05-19	1938	503129.7800000...	
5	2026-05-08	1939	501421.8700000...	
6	2026-04-22	1939	492340.7700000...	
7	2026-04-20	1948	512261.4499999...	
8	2026-05-14	1952	505992.3600000...	
9	2026-04-24	1954	521197.5400000...	
10	2026-04-21	1956	512332.4900000...	

Tiempo transcurrido
281 ms

Tiempo de ranura consumido ⓘ
61 ms

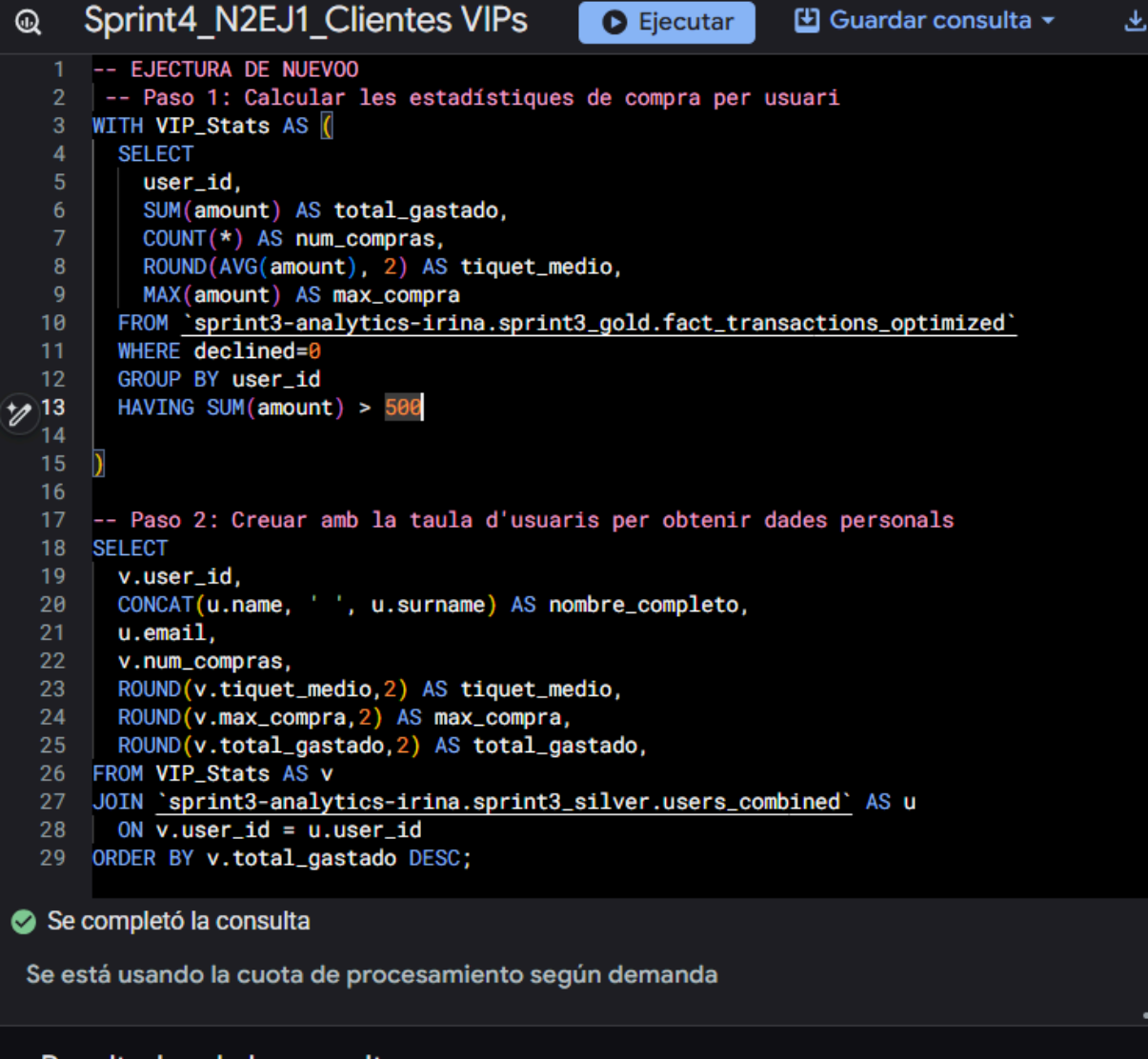
Bytes generados y movidos ⓘ
1.03 KB

Bytes volcados al disco ⓘ
0 B ⓘ

He implementado la Vista Materializada (mv_daily_sales) para aplicar Smart Caching. Al precalcular y almacenar físicamente las ventas diarias, se evita escaneos repetitivos de la tabla base, reduciendo los costos y el tiempo de respuesta para múltiples usuarios simultáneos.

NIVEL 2

EJERCICIO 1



```
1  -- EJECTURA DE NUEVOO
2  -- Paso 1: Calcular les estadístiques de compra per usuari
3  WITH VIP_Stats AS (
4      SELECT
5          user_id,
6          SUM(amount) AS total_gastado,
7          COUNT(*) AS num_compras,
8          ROUND(AVG(amount), 2) AS tiquet_medio,
9          MAX(amount) AS max_compra
10     FROM `sprint3-analytics-irina.sprint3_gold.fact_transactions_optimized`
11     WHERE declined=0
12     GROUP BY user_id
13     HAVING SUM(amount) > 500
14 )
15
16
17 -- Paso 2: Creuar amb la taula d'usuaris per obtenir dades personals
18 SELECT
19     v.user_id,
20     CONCAT(u.name, ' ', u.surname) AS nombre_completo,
21     u.email,
22     v.num_compras,
23     ROUND(v.tiquet_medio,2) AS tiquet_medio,
24     ROUND(v.max_compra,2) AS max_compra,
25     ROUND(v.total_gastado,2) AS total_gastado,
26 FROM VIP_Stats AS v
27 JOIN `sprint3-analytics-irina.sprint3_silver.users_combined` AS u
28     ON v.user_id = u.user_id
29 ORDER BY v.total_gastado DESC;
```

Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta								
Información del trabajo		Resultados	Visualización	JSON	Detalles de la ejecución		Gráfico de ejecución	
Fila	user_id	nombre_completo	email	num_compras	tiquet_medio	max_compra	total_gastado	
1	289	Dxwgi Hwcru	dxwgi.hwcru@example.com	91	446.91	554.71	40669.16	
2	318	Bnyr Astuw	bnyr.astuw@example.com	86	449.0	615.51	38614.39	
3	455	Fwfl Phylgnmhnd	fwfl.phylgnmhnd@example.com	66	522.59	724.44	34491.27	
4	417	Fqatil Rdwcjvoa	fqatil.rdwcjvoa@example.com	73	421.87	675.78	30796.3	
5	185	Molly Gilliam	donec@outlook.couk	107	274.72	659.31	29394.79	
6	285	Brcxpmj Lnpna	brcxpmj.lnpna@example.com	70	417.85	630.52	29249.51	
7	193	Minerva Wilkins	aliquam.adipiscing@yahoo.edu	67	380.37	646.0	25485.05	
8	454	Sfzzoh Xgvfridxs	sfzzoh.xgvfridxs@example.com	78	304.0	604.9	23711.9	
9	349	Yxxcjk Fnceqbkmmt	yxxcjk.fnceqbkmmt@example.c...	53	443.48	675.05	23504.47	
10	317	Qikcvc Nvfciqdh	qikcvc.nvfciqdh@example.com	54	351.69	600.77	18991.45	
11	388	Rbjupnac Cfjykdxd	rbjupnac.cfjykdxd@example.com	62	297.3	479.03	18432.84	
12	319	Fxtg Eedgl	fxtg.eedgl@example.com	62	286.62	576.22	17770.39	

Bytes procesados	2.55 MB
Bytes facturados	20 MB

He implementado una CTE llamada (VIP_Stats)para pre-agregar el comportamiento de compra (gasto total, ticket medio y compra máxima) filtrando solo a los clientes con más de 500€ de gasto. Posteriormente, he realizado un JOIN con la dimensión de usuarios para enriquecer el informe con sus datos de contacto y he consolidado el nombre y apellido en un solo campo dinámico.

EJERCICIO 2

Sprint4_N2EJ2_Análisis de tend...

Ejecutar

Guardar

```
1
2 -- creación Windows Function
3 SELECT
4     sales_date AS fecha,
5     ROUND (total_sales) AS Ventas_hoy,
6     ROUND(LAG(total_sales) OVER (ORDER BY sales_date),2) AS Ventas_Ayer,
7     ROUND(
8         (total_sales - LAG(total_sales) OVER (ORDER BY sales_date))
9         / LAG(total_sales) OVER (ORDER BY sales_date) * 100,
10        2
11    ) AS Diff_Percentual
12 FROM `sprint3-analytics-irina.sprint3_gold.mv_daily_sales`
13 ORDER BY sales_date;
14
15
16
```

Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Información del trabajo

Resultados

Visualización

JSON

Detalle

Fila	fecha	Ventas_hoy	Ventas_Ayer	Diff_Percentual
1	2026-04-09	509588.0	null	null
2	2026-04-10	536468.0	509588.43	5.27
3	2026-04-11	510591.0	536467.93	-4.82
4	2026-04-12	504498.0	510590.71	-1.19
5	2026-04-13	514183.0	504497.92	1.92
6	2026-04-14	525617.0	514182.79	2.22
7	2026-04-15	525739.0	525617.25	0.02
8	2026-04-16	518556.0	525739.04	-0.88

Bytes procesados

800 B

Bytes facturados

10 MB

He realizado este análisis de tendencias diarias consultando directamente la Vista Materializada (mv_daily_sales), para tener una respuesta instantánea y evitar gastos por consulta. Para comparar el rendimiento entre días consecutivos (Day-over-Day Growth), he utilizado la función de navegación LAG, que permite acceder de forma eficiente al valor de la fila precedente en la serie temporal sin necesidad de realizar un JOIN. Sobre este valor,

he aplicado la fórmula de variación porcentual redondeada a dos decimales para mostrar crecimientos o caídas en las ventas.

EJERCICIO 3

Sprint4_N2EJ3_Total acumulad... Ejecutar

```

1
2 SELECT
3   sales_date AS Fechas,
4   ROUND(total_sales, 2) AS Ventas_Dia,
5   ROUND(
6     SUM(total_sales) OVER (
7       PARTITION BY EXTRACT(YEAR FROM sales_date)
8       ORDER BY sales_date
9       ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
10    ),
11    2
12  ) AS Ventas_Acumuladas_YTD
13 FROM `sprint3-analytics-irina.sprint3_gold.mv_daily_sales`
14 ORDER BY sales_date;
15

```

✓ Se completó la consulta
Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Información del trabajo	Resultados	Visualización	JSON
Fila	Fechas	Ventas_Dia	Ventas_Acumula...
1	2026-04-09	509588.43	509588.43
2	2026-04-10	536467.93	1046056.36
3	2026-04-11	510590.71	1556647.07
4	2026-04-12	504497.92	2061144.99
5	2026-04-13	514182.79	2575327.78
6	2026-04-14	525617.25	3100945.03
7	2026-04-15	525739.04	3626684.07
8	2026-04-16	543556.9	4170240.97

✓ Esta consulta procesará 800 B cuando se ejecute.

He implementado un informe de totales acumulados (Year-To-Date) utilizando funciones de ventana sobre la Vista Materializada mv_daily_sales. Usé PARTITION BY EXTRACT(YEAR FROM Data), que permite que el cálculo de la suma progresiva se reinicie automáticamente con cada nuevo año natural. Lo combiné con el marco de ventana ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW, para ver la curva de crecimiento anual de las ventas sin incurrir en costes de procesamiento adicionales sobre los datos base.

EJERCICIO 4

```
1  -- Fidelización y Valor del Cliente
2  WITH Primeras_Compras AS (
3      SELECT
4          user_id,
5          Sales_Timestamp,
6          amount,
7          ROW_NUMBER() OVER (
8              PARTITION BY user_id
9              ORDER BY Sales_Timestamp
10         ) AS numero_compra
11  FROM `sprint3-analytics-irina.sprint3_gold.fact_transactions_optimized`
12  WHERE declined=0
13  QUALIFY numero_compra <= 3
14  )
15  SELECT
16      u.user_id,
17      CONCAT(u.name, ' ', u.surname) AS nombre_completo,
18      u.email,
19      MAX(CASE WHEN pc.numero_compra = 3 THEN pc.Sales_Timestamp END) AS fecha_3a_compra,
20      MAX(CASE WHEN pc.numero_compra = 3 THEN pc.amount END) AS importe_3a_compra,
21      ROUND(AVG(pc.amount), 2) AS promedio_3_primeras
22  FROM Primeras_Compras AS pc
23  JOIN `sprint3-analytics-irina.sprint3_silver.users_combined` AS u
24      ON pc.user_id = u.user_id
25  GROUP BY u.user_id, nombre_completo, u.email
26  ORDER BY promedio_3_primeras DESC;
```

Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta								Chat	Guardar l
Información del trabajo		Resultados	Visualización	JSON	Detalles de la ejecución		Gráfico de ejecución		
Fila	user_id	nombre_completo	email	fecha_3a_compra	importe_3a_com...	promedio_3_prim...			
1	285	Brcxprnj Lnpna	brcxprnj.lnpna@example.com	2026-04-11 07:25:06.504893 UTC	630.52	630.52			
2	4053	Yieyga Szmwdfhu	yieyga.szmwdfhu@example.com	2026-04-13 07:25:06.504893 UTC	570.46	596.28			
3	4704	Cdkmoo Gaqthofe	cdkmoo.gaqthofe@example.com	2026-04-19 07:25:06.504893 UTC	683.49	596.0			
4	414	Djphbb Cliwfto	djphbb.cliwfto@example.com	2026-04-14 07:25:06.504893 UTC	578.42	586.07			
5	455	Fwfill Phylgnmhnd	fwfill.phylgnmhnd@example.com	2026-04-11 07:25:06.504893 UTC	678.61	581.78			
6	3102	Mzncuv Wagrixdz	mzncuv.wagrixdz@example.com	2026-04-12 07:25:06.504893 UTC	476.62	560.08			
7	2977	Gvmjlk Zglxqeg	gvmjlk.zglxqeg@example.com	2026-04-14 07:25:06.504893 UTC	507.67	551.51			
8	2079	Hihige Rplahrbx	hihige.rplahrbx@example.com	2026-04-18 07:25:06.504893 UTC	342.35	543.95			
9	1692	Xqfnef Mtyrnkhv	xqfnef.mtyrnkhv@example.com	2026-04-14 07:25:06.504893 UTC	488.29	543.53			
10	909	Ddtrhr Npgaulk	ddtrhr.npgaulk@example.com	2026-04-17 07:25:06.504893 UTC	551.49	536.43			
11	3527	Whpxjm Xpzaijen	whpxjm.xpzaijen@example.com	2026-04-30 07:25:06.504893 UTC	415.08	525.11			
12	486	Soqn Ojibnqgluf	soqn.ojibnqgluf@example.com	2026-04-11 07:25:06.504893 UTC	427.42	522.49			
13	3369	Jruzfm Nchaxfrv	jruzfm.nchaxfrv@example.com	2026-04-16 07:25:06.504893 UTC	579.44	521.9			

En este ejercicio de fidelización, he implementado una estrategia de filtrado temprano utilizando la función de ventana `ROW_NUMBER()` para asignar un orden cronológico a las transacciones de cada usuario. A través de la cláusula `QUALIFY`, se ha restringido los datos a las tres primeras compras antes de ejecutar el `JOIN` con la dimensión de usuarios.

Finalmente, he aplicado funciones de agregado como AVG para obtener el gasto promedio inicial y MAX combinada con la lógica CASE para identificar específicamente los detalles de fecha e importe del tercer hito de compra, permitiendo segmentar a los clientes según su valor para el negocio.

NIVEL 3

EJERCICIO 1

```
Sprint4_N3EJ1_Desanidamient... Ejecutar Guardar consulta
```

```
1 CREATE OR REPLACE TABLE `sprint3-analytics-irina.sprint3_gold.dim_transactions_flat` AS
2 SELECT
3     t.transaction_id,
4     t.Sales_Timestamp AS trans_timestamp,
5     t.amount AS total_ticket,
6     pid AS product_sku,
7     p.name AS product_name,
8     p.price AS product_price
9 FROM `sprint3-analytics-irina.sprint3_gold.fact_transactions_optimized` AS t
10 CROSS JOIN UNNEST(
11     (SELECT ARRAY_AGG(DISTINCT x) FROM UNNEST(t.product_ids_array) AS x)
12 ) AS pid
13 JOIN `sprint3-analytics-irina.sprint3_silver.products_clean` AS p
14 ON CAST(pid AS INT64) = p.product_id;
```

Esta consulta procesará 7.09 MB cuando se ejecute.

Esquema	Detalles	Vista previa	Explorador de tablas	Vista previa	Estadísticas	Linaje	Perfil de datos	Calidad de los datos
transaction_id	trans_timestamp	total_ticket	product_sku	product_name	product_price			
68	89217BC9-8C96-4765-AC1B-85...	2026-04-09 07:25:06.504893 UTC	176.31	2	Tarly Stark	9.24		
69	1A68760D-E05C-4E7D-979B-BF...	2026-04-09 07:25:06.504893 UTC	180.46	2	Tarly Stark	9.24		
70	3A7A23FB-6482-46CE-AF45-17...	2026-04-09 07:25:06.504893 UTC	268.28	2	Tarly Stark	9.24		
71	A68111D2-8C81-4B6C-80FC-BD...	2026-04-09 07:25:06.504893 UTC	82.1	2	Tarly Stark	9.24		
72	75E0F9FD-2105-46EB-B74B-AA...	2026-04-09 07:25:06.504893 UTC	289.71	2	Tarly Stark	9.24		
73	266AEDA6-CD7F-4DC2-B0B3-8...	2026-04-09 07:25:06.504893 UTC	101.2	2	Tarly Stark	9.24		
74	D98C1B27-8D73-4C9B-9BDC-EF...	2026-04-09 07:25:06.504893 UTC	466.02	2	Tarly Stark	9.24		
75	95D56335-F19C-463A-B8B8-C8...	2026-04-09 07:25:06.504893 UTC	290.4	2	Tarly Stark	9.24		
76	90F8C2B7-E47F-479A-8422-F6...	2026-04-09 07:25:06.504893 UTC	282.73	2	Tarly Stark	9.24		
77	13F7591D-1EBE-4B9E-9B3C-AA...	2026-04-09 07:25:06.504893 UTC	530.02	2	Tarly Stark	9.24		
78	0630BE76-04AB-4E1B-A94C-59...	2026-04-09 07:25:06.504893 UTC	350.47	3	duel tourney Lannister	171.13		
79	93260997-0833-4F2A-87FC-F67...	2026-04-09 07:25:06.504893 UTC	511.96	3	duel tourney Lannister	171.13		
80	37D8BE19-A164-4EF3-857A-F1...	2026-04-09 07:25:06.504893 UTC	339.62	3	duel tourney Lannister	171.13		
81	D6EF0146-CB5C-4BE6-B0F0-A9...	2026-04-09 07:25:06.504893 UTC	410.27	3	duel tourney Lannister	171.13		
82	86358621-6CCD-4640-8566-E5...	2026-04-09 07:25:06.504893 UTC	397.32	3	duel tourney Lannister	171.13		
83	68293A6F-6DDC-4525-9F8B-ED...	2026-04-09 07:25:06.504893 UTC	171.13	3	duel tourney Lannister	171.13		
84	CDEFAD1B-3FA8-42C9-8E34-AD...	2026-04-09 07:25:06.504893 UTC	366.75	3	duel tourney Lannister	171.13		
85	6EE2495C-6E64-4D17-99CC-35...	2026-04-09 07:25:06.504893 UTC	406.35	3	duel tourney Lannister	171.13		
86	2442068D-52C4-494B-84A7-AF...	2026-04-09 07:25:06.504893 UTC	427.16	3	duel tourney Lannister	171.13		

Resultados por página: 50 51 - 100 de 253391

Bytes procesados	7.09 MB
Bytes facturados	20 MB

Para cumplir con el requerimiento del equipo de ventas, he desnormalizado la estructura de las transacciones creando una tabla plana (dim_transactions_flat). He utilizado la cláusula CROSS JOIN UNNEST para 'explotar' el array de identificadores de productos, convirtiendo

cada elemento en una fila individual e independiente. Posteriormente, he realizado un JOIN con el catálogo maestro de productos (products_clean). Así se elimina la complejidad de los datos anidados, permitiendo que cualquier usuario de negocio pueda generar consultas de ventas por artículo.

EJERCICIO 2

Sprint4_N3EJ2_Ranking de ven...

Ejecutar

Guar

```
1
2 SELECT
3 product_sku
4   product_name,
5   COUNT(*) AS unidades_vendidas
6 FROM `sprint3-analytics-irina.sprint3_gold.dim_transactions_flat`
7 GROUP BY product_name, product_sku
8 ORDER BY unidades_vendidas DESC
9 LIMIT 5;
10
```

✓ Esta consulta procesará 1.93 MB cuando se ejecute.
Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Información del trabajo	Resultados	Visualización	JSON	D
Fila	product_name	unidades_vendidas		
1	52	2654		
2	29	2635		
3	21	2609		
4	16	2608		
5	66	2601		

Bytes procesados	1.93 MB
Bytes facturados	10 MB
Milisegundos de ranura	36

Para generar el ranking de ventas, he consultado directamente la tabla desnormalizada (dim_transactions_flat). Al ser una estructura plana donde cada registro equivale a un producto vendido, solo ha sido necesario realizar una agregación estándar mediante GROUP BY y COUNT(*). La tabla (dim_transactions_flat) permite obtener el Top 5 de productos de forma rápida y eficiente sin necesidad de procesos de desanidamiento adicionales.

EJERCICIO 3

Sprint4_N3EJ3_Automatizació...
Ejecutar
Guardar consulta
Descargar
Compartir

```

1
2 -- función de IVA (21%) de forma persistente
3 CREATE OR REPLACE FUNCTION `sprint3-analytics-irina.sprint3_gold.calculate_tax`(amount FLOAT64)
4 RETURNS FLOAT64
5 AS (
6   amount * 1.21
7 );
8
9 -- La tabla plana automatizada con cálculo de impuestos
10
11 CREATE OR REPLACE TABLE `sprint3-analytics-irina.sprint3_gold.dim_transactions_flat` AS
12 SELECT
13   t.transaction_id,
14   t.Sales_Timestamp AS trans_timestamp,
15   t.amount AS total_ticket,
16   p.product_id AS product_sku,
17   p.name AS product_name,
18   p.price AS product_price,
19   `sprint3-analytics-irina.sprint3_gold.calculate_tax`(p.price) AS product_price_tax_inc -- Aplicamos la UDF aquí.
20 FROM `sprint3-analytics-irina.sprint3_gold.fact_transactions_optimized` AS t
21 CROSS JOIN UNNEST(t.product_ids_array) AS pid
22 JOIN `sprint3-analytics-irina.sprint3_silver.products_clean` AS p
23   ON CAST(pid AS INT64) = p.product_id;

```

Se completó la consulta
Se está usando la cuota de procesamiento según demanda

Todos los resultados

Tiempo transcurrido
3 s

Declaraciones procesadas
2

Estado del trabajo
SUCCESS

Bytes procesados	0 B
Bytes facturados	0 B
Milisegundos de ranura	0

He implementado una User Defined Function (UDF) persistente llamada calculate_tax, que centraliza la lógica del cálculo del IVA al 21% y evita el uso de valores fijos (hardcoded) en las consultas. Esta función se ha integrado en la actualización de la tabla (dim_transactions_flat), permitiendo generar de forma automática la columna

product_price_tax_inc mientras se realiza el aplanamiento de los productos mediante UNNEST.